

Análise e Implementação de Redução Transitiva em Grafos Direcionados Gerais

Ana Clara Lonczynski  [Pontifícia Universidade Católica de Minas Gerais | anaclara.lon@hotmail.com]

Arthur Carvalho Rodrigues  [Pontifícia Universidade Católica de Minas Gerais | arthurcarvalhorodrigues2409@gmail.com]

Bruno Rafael Santos Oliveira  [Pontifícia Universidade Católica de Minas Gerais | brunorafaelso04@gmail.com]

Débora Luiza de Paula Silva  [Pontifícia Universidade Católica de Minas Gerais | debora.paula05@gmail.com]

Mariana Almeida Mendonça  [Pontifícia Universidade Católica de Minas Gerais | marianaalmeidafga@gmail.com]

Matheus Eduardo Campos Soares  [Pontifícia Universidade Católica de Minas Gerais | matheuseducs@gmail.com]

Rayssa Mell de Souza Silva  [Pontifícia Universidade Católica de Minas Gerais | rayssamellsouza@gmail.com]

Thiago Pereira de Oliveira  [Pontifícia Universidade Católica de Minas Gerais | thiago.p.oliveira@protonmail.com]

 Pontifícia Universidade Católica de Minas Gerais, Av. Dom José Gaspar, 500, Coração Eucarístico, Belo Horizonte, MG, 30535-901, Brazil.

Abstract. Este trabalho apresenta uma abordagem para redução transitiva em grafos direcionados, buscando remover arestas redundantes sem alterar a relação de alcançabilidade entre os vértices. A solução combina caminhamentos em profundidade, identificação de Componentes Fortemente Conexas pelo algoritmo de Tarjan, condensação em DAG e heurísticas de busca interna para tratar grafos com ciclos. Diferentes estratégias são comparadas quanto ao custo computacional e à quantidade de arestas preservadas, destacando os limites entre redução exata, segurança estrutural e compressão experimental.

Keywords: Grafos direcionados; redução transitiva; alcançabilidade; componentes fortemente conexas; algoritmo de Tarjan; DAG; heurísticas; DFS.

1 Introdução

Algoritmos baseados em grafos são fundamentais para a resolução de problemas complexos na Ciência da Computação. Um problema clássico e de grande relevância prática nesse domínio é a *redução transitiva*: dado um grafo dirigido $G = (V, E)$, busca-se obter um subgrafo $G' = (V, E')$, com $E' \subseteq E$, que preserve integralmente a atingibilidade entre todos os pares de vértices, ao mesmo tempo em que minimiza a cardinalidade do conjunto de arestas $|E'|$ Aho *et al.* [1972]. A atingibilidade de um vértice representa o conjunto de nós alcançáveis a partir dele; dessa forma, eliminam-se caminhos redundantes sem comprometer o alcance lógico entre os vértices.

Este trabalho investiga o problema da redução transitiva em grafos dirigidos gerais, incluindo estruturas complexas contendo ciclos. A abordagem central combina a identificação de Componentes Fortemente Conexas, ou *Strongly Connected Components* (SCCs), via algoritmo de Tarjan [1972], com a redução da macroestrutura condensada em um Grafo Acíclico Dirigido, ou *Directed Acyclic Graph* (DAG), sobre o qual o Teorema de Aho, Garey e Ullman assegura computabilidade em tempo polinomial. O tratamento do interior das SCCs, por sua vez, exige o uso de heurísticas baseadas em caminhamento, dado que o problema de minimização exata para essas estruturas com ciclos é NP-difícil Khuller *et al.* [1995].

Para avaliar empiricamente essas estratégias, foram desenvolvidas diferentes variantes algorítmicas em linguagem C++, variando desde baselines de força bruta até heurísticas otimizadas para componentes fortemente conexas. O desenvolvimento priorizou o uso de listas de adjacência como

estrutura de dados base, visando garantir a eficiência assintótica dos caminhamentos realizados. Adicionalmente, analisou-se o impacto teórico de estender o problema para grafos não direcionados.

Neste contexto, as principais contribuições deste trabalho são resumidas da seguinte forma: (i) uma análise da aplicação de redução transitiva em grafos direcionados gerais contendo ciclos; (ii) a avaliação de cinco abordagens algorítmicas, de um *baseline* de força bruta a estratégias com SCCs, DAGs e busca interna, incluindo anéis apenas como contraponto demonstrativo por não preservar $E' \subseteq E$; e (iii) uma avaliação empírica detalhada do tempo de execução e da eficiência de compressão dessas abordagens.

2 Conceitos Fundamentais

Esta seção resume os conceitos diretamente utilizados na implementação: redução transitiva, arestas redundantes, caminhamentos de verificação e decomposição em componentes fortemente conexas.

2.1 Redução transitiva e redundância

Seja $G = (V, E)$ um grafo direcionado. A redução transitiva busca obter um subgrafo $G' = (V, E')$, com $E' \subseteq E$, que preserve a mesma relação de atingibilidade de G . Neste trabalho, uma aresta (u, v) é considerada redundante quando existe, no grafo corrente sem essa aresta, um caminho alternativo de u até v . Assim, uma remoção só é aceita quando não altera os vértices alcançáveis a partir de u .

Na implementação, esse critério é aplicado sobre uma

cópia modificável do grafo. As arestas candidatas são percorridas e, para cada uma delas, executa-se uma busca de atingibilidade ignorando temporariamente a própria aresta avaliada. Caso o destino ainda seja alcançável, a aresta é removida de G' . Dessa forma, a decisão de remoção não depende apenas da existência da aresta no grafo original, mas da preservação da alcançabilidade no estado corrente da redução.

Em DAGs, essa remoção pode ser feita diretamente pela identificação de caminhos indiretos. Em grafos direcionados gerais, a presença de ciclos exige separar a estrutura global acíclica das regiões fortemente conexas.

2.2 Caminhamentos e componentes fortemente conexas

A verificação de redundância é feita por caminhamentos em profundidade. Para cada aresta candidata (u, v) , a busca testa se v permanece alcançável a partir de u sem utilizar diretamente essa aresta. Esse mesmo princípio é usado para validar se o grafo reduzido preserva a atingibilidade do grafo original.

A implementação utiliza listas de adjacência como estrutura de representação do grafo. Essa escolha se justifica por duas razões complementares: (i) os algoritmos de caminho percorrem diretamente os vizinhos de cada vértice, e a lista de adjacência permite essa iteração em tempo proporcional ao grau do vértice, sem inspecionar entradas ausentes; e (ii) grafos esparsos — como os utilizados nos experimentos — têm $|E| \ll |V|^2$, de modo que a lista de adjacência consome $O(V + E)$ de memória, enquanto uma matriz de adjacência exigiria $O(V^2)$ mesmo para arestas inexistentes. Esse compromisso entre custo de memória e custo de acesso favorece a lista de adjacência para os caminhamentos DFS que fundamentam todas as variantes implementadas.

Para tratar ciclos, o grafo é decomposto em componentes fortemente conexas pelo algoritmo de Tarjan. Cada componente é contraída em um supervértice, formando um DAG de condensação. As arestas entre componentes são reduzidas nessa estrutura acíclica, enquanto as arestas internas às SCCs são preservadas ou avaliadas por buscas restritas à própria componente, conforme a variante implementada.

3 Metodologia

A metodologia adotada organiza a redução transitiva a partir de caminhamentos no grafo, separando o tratamento da estrutura acíclica global do tratamento das regiões cíclicas. Para isso, as variantes implementadas utilizam a identificação de componentes fortemente conexas, a condensação do grafo em DAG e buscas de atingibilidade para decidir a remoção de arestas.

3.1 Hipótese e representação

A hipótese central é que é possível realizar a redução transitiva de um grafo direcionado geral utilizando apenas caminhamentos. Dado $G = (V, E)$, busca-se um subgrafo

$G' = (V, E')$, com $E' \subseteq E$, que preserve a atingibilidade entre todos os pares de vértices e minimize $|E'|$.

Para isso, adotam-se três hipóteses operacionais:

1. uma aresta (u, v) é redundante se uma DFS encontra um caminho alternativo de u até v sem utilizá-la;
2. condensar os Componentes Fortemente Conexos em um DAG por meio do algoritmo de Tarjan reduz o espaço de busca das arestas entre componentes;
3. uma DFS restrita ao interior de cada componente permite remover arestas internas redundantes sem criar arestas artificiais.

O grafo foi representado por listas de adjacência, pois as variantes dependem da iteração sobre vizinhos e da execução de caminhamentos. Essa representação evita o custo de memória de uma matriz de adjacência e permite que buscas em profundidade percorram apenas as arestas efetivamente existentes. A redução é aplicada sobre uma cópia modificável do grafo original, preservando a entrada para validação posterior.

3.2 Variantes implementadas

Foram avaliadas cinco abordagens algorítmicas, definidas de acordo com a topologia do grafo de entrada. Essas variantes se diferenciam pelo escopo de atuação: algumas reduzem a macroestrutura acíclica obtida pela condensação em DAG, enquanto outras tratam diretamente as arestas internas às componentes cíclicas.

- **Baseline (--reduce):** espera-se que identifique e remova qualquer aresta cuja eliminação não altere a matriz de alcançabilidade global. Como avalia o grafo de forma linear e oculta arestas individualmente através de uma varredura baseada em um *snapshot* estático das conexões originais, o seu comportamento serve como gabarito estrito de correteude.
- **Otimizado para DAGs (--reduce-dag):** restrito a estruturas acíclicas. Com base no Teorema de Aho-Garey-Ullman, converge para uma redução transitiva única e ótima G' , linearizando cadeias causais através da busca por caminhos indiretos.
- **Conservador (--reduce-optimized-conservative):** atua exclusivamente nas arestas inter-componentes do DAG de condensação G^{cd} . O comportamento projetado é priorizar a segurança da conectividade entre componentes, preservando integralmente o interior das SCCs identificadas por Tarjan em prol da integridade dos ciclos originais.
- **Busca Interna (--reduce-internal-search):** projetado como uma heurística de aproximação para o problema do Grafo Equivalente Mínimo, ou *Minimum Equivalent Graph* (MEG), que é NP-difícil. O comportamento esperado é a redução de arestas internas por meio de uma DFS modificada. Esta busca é controlada por uma restrição baseada no vetor `compOfVertex`, que limita o caminhamento estritamente aos nós pertencentes ao mesmo identificador de componente do vértice de origem, impedindo o uso de caminhos externos à SCC avaliada para justificar remoções internas.

- **Abordagem de Anéis** (`--reduce-optimized-rings`): incluída exclusivamente como contraponto demonstrativo de compressão limite, sem pretensão de satisfazer a definição de redução transitiva estrita. O comportamento consiste na substituição do interior de cada SCC por um ciclo direcionado simples perfeito. Embora preserve a atingibilidade global, essa estratégia introduz arestas sintéticas que não pertencem ao grafo original, violando a premissa $E' \subseteq E$. Sua inclusão nos experimentos serve apenas para delimitar o patamar máximo de compressão possível quando essa restrição é relaxada, e seus resultados não devem ser comparados diretamente às variantes válidas.

3.3 Procedimento de remoção, ciclos e validação

O procedimento de remoção parte de uma cópia modificável do grafo original, denotada por G' . As arestas candidatas são obtidas a partir de um *snapshot* de E , enquanto as remoções são aplicadas progressivamente em G' . Para cada aresta (u, v) avaliada, a implementação testa se v permanece alcançável a partir de u no grafo reduzido corrente, desconsiderando a própria aresta candidata. Caso esse caminho alternativo exista, a aresta é removida; caso contrário, ela é preservada.

Algorithm 1 Redução Transitiva com Busca Interna em SCCs

Require: Grafo direcionado $G = (V, E)$
Ensure: Grafo reduzido $G' = (V, E')$

```

1:  $components \leftarrow \text{ExecutarTarjan}(G)$ 
2:  $compOfVertex \leftarrow \text{MapearVerticesParaComponentes}(components)$ 
3:  $G' \leftarrow \text{CopiarGrafoOriginal}(G)$ 
4:  $E_{snapshot} \leftarrow \text{ObterListaDeArestas}(G)$ 
5: for cada aresta  $(u, v)$  em  $E_{snapshot}$  do
6:   if  $compOfVertex[u] = compOfVertex[v]$  then
7:      $targetComp \leftarrow compOfVertex[u]$ 
8:     if  $\text{CaminhoAlternativoExisteEmSCC}(G', u, v, targetComp)$ 
9:       then
10:         $\text{RemoverAresta}(G', u, v)$ 
11:     end if
12:   end if
13: end for
14: return  $G'$ 
    
```

No caso das componentes fortemente conexas, o vetor *compOfVertex* restringe a busca ao identificador da componente de origem. Essa restrição impede que caminhos externos à SCC sejam usados para justificar a remoção de arestas internas. Assim, o tratamento dos ciclos ocorre localmente: arestas internas são avaliadas apenas dentro da própria componente, enquanto a macroestrutura entre componentes é tratada no DAG de condensação pelas variantes específicas.

A validação da redução é feita pela comparação da atingibilidade antes e depois da execução. Para cada vértice de

origem, são obtidos os conjuntos de vértices alcançáveis em G e em G' . A saída de uma variante é considerada válida apenas quando esses conjuntos coincidem para todos os vértices, garantindo que a redução removeu arestas sem alterar a relação de alcançabilidade do grafo original.

4 Análise de Custo

A análise de custo toma como referência o Algoritmo 1, apresentado na metodologia, por ser a variante que combina decomposição em SCCs e buscas internas para remoção de arestas. O custo total decorre de três etapas principais: inicialização das estruturas, varredura das arestas candidatas e execução das buscas alternativas.

Inicialização (linhas 1–4): a identificação das SCCs pelo algoritmo de Tarjan, o mapeamento dos vértices para seus respectivos identificadores de componente, a clonagem do grafo e a construção do *snapshot* de arestas operam em tempo linear $O(V + E)$. Em particular, o preenchimento do vetor *compOfVertex*, realizado após a identificação das componentes, consome $O(V)$.

Filtro estrutural (linhas 5–6): o laço principal percorre as arestas presentes em $E_{snapshot}$, cujo tamanho é $|E|$. A condicional da linha 6 verifica a pertinência dos extremos da aresta à mesma componente em tempo constante $O(1)$ por iteração, permitindo distinguir arestas internas às SCCs de arestas intercomponentes.

Pior caso teórico (linha 8): no pior caso, o grafo consiste de uma única SCC global, isto é, $V_i = V$ e $E_i = E$. Nesse cenário, cada DFS modificada pode inspecionar toda a estrutura do grafo. Como até $|E|$ arestas podem acionar essa busca, a complexidade limita-se superiormente por $O(E \cdot (V + E))$. Em grafos densos, nos quais $E \approx V^2$, esse limite se torna $O(V^4)$, igualando o patamar assintótico do *baseline --reduce*.

Caso geral prático: em instâncias fragmentadas em múltiplas SCCs, a restrição por componente limita a DFS ao escopo de *targetComp*. Assim, o custo passa a depender do somatório das subestruturas internas avaliadas:

$$O\left(\sum_{i=1}^k |E_i| \cdot (|V_i| + |E_i|)\right), \quad (1)$$

onde V_i e E_i são, respectivamente, os vértices e as arestas internos da i -ésima SCC. Essa formulação evidencia que, quanto mais fragmentado o grafo estiver em componentes menores, menor tende a ser o custo prático das buscas internas em relação ao pior caso global.

As demais variantes estabelecem patamares de custo distintos. O *baseline --reduce* pode atingir $O(E \cdot (V + E))$, por testar arestas individualmente no grafo. A *--reduce-dag* realiza uma DFS por vértice, operando em $O(V \cdot (V + E))$. A abordagem conservadora reduz apenas o macro-DAG, limitando-se a $O(V_c \cdot (V_c + E_c))$, onde V_c e E_c representam os vértices e arestas do grafo de condensação. Por fim, o modelo de anéis reconecta os componentes internos em tempo linear $O(V + E)$ após a identificação das SCCs, mas não caracteriza redução transitiva estrita quando introduz arestas sintéticas.

5 Resultados e Análise Empírica

5.1 Configuração Experimental

A avaliação empírica foi conduzida para comparar, de forma objetiva, o desempenho das variantes implementadas quanto ao tempo de execução, à capacidade de redução de arestas e à preservação da atingibilidade do grafo original. Os testes foram executados em ambiente Linux, em uma máquina equipada com processador AMD Ryzen. Os algoritmos foram compilados com g++, compatível com o padrão ISO C++17, utilizando a flag de otimização -O2 e os alertas de compilação -Wall e -Wextra.

O protocolo experimental submeteu cada variante a 10 execuções por instância, considerando o tempo médio de processamento e a variação observada. Foram utilizadas duas instâncias principais: uma instância geral com SCCs, composta por 300 vértices e 924 arestas, e uma instância acíclica, composta por 300 vértices e 1137 arestas. A instância com SCCs foi usada para avaliar as variantes aplicáveis a grafos direcionados gerais, enquanto a instância DAG foi reservada à variante --reduce-dag. Após cada redução, a atingibilidade foi validada pela comparação dos conjuntos de vértices alcançáveis no grafo original e no grafo reduzido. Ressalta-se que a avaliação se restringiu a duas instâncias de escala fixa, com fins educativos e comparativos entre as variantes. Uma avaliação de escalabilidade com instâncias de maior porte e densidades variadas constituiria extensão natural deste trabalho.

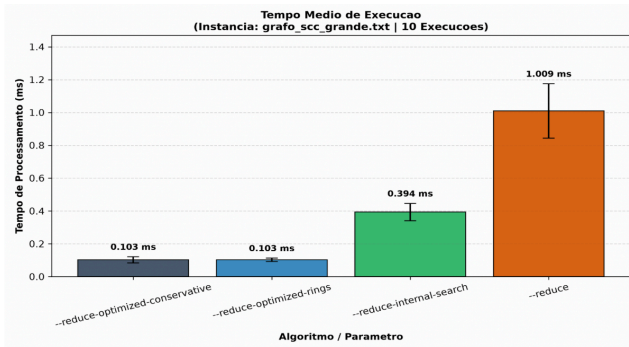


Figure 1. Tempo médio de execução dos algoritmos na instância geral.

O Gráfico 1 evidencia a diferença de custo entre as estratégias. O *baseline* --reduce apresentou o pior desempenho, com tempo médio de 1,009 ms, resultado esperado por testar a redundância das arestas de forma direta e repetitiva. A variante --reduce-internal-search reduziu esse custo para 0,394 ms ao restringir as buscas ao interior das componentes fortemente conexas, mantendo a mesma lógica de preservação de atingibilidade. Já as abordagens --reduce-optimized-conservative e --reduce-optimized-rings ficaram no menor patamar de tempo, ambas próximas de 0,103 ms, pois evitam buscas internas sucessivas em larga escala.

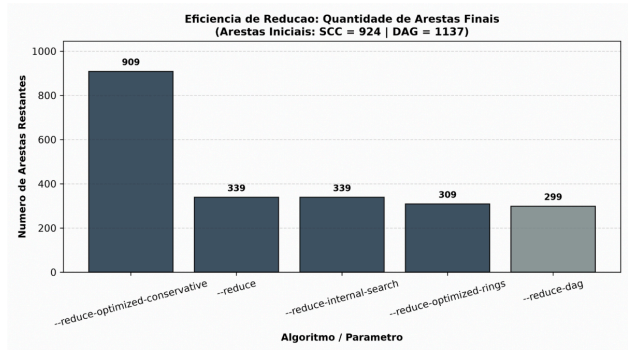


Figure 2. Eficiência de redução: quantidade de arestas finais após a compressão.

O Gráfico 2 mostra que velocidade isolada não implica boa redução. A abordagem conservadora manteve 909 das 924 arestas originais, removendo apenas uma fração pequena do grafo. Embora seja rápida, sua compressão é limitada por preservar integralmente o interior das SCCs. Em contraste, o *baseline* --reduce e a variante --reduce-internal-search reduziram o grafo para 339 arestas finais, preservando a condição de subgrafo estrito. Assim, na instância com SCCs, a busca interna alcançou a mesma redução do *baseline* com custo substancialmente menor.

A variante --reduce-optimized-rings obteve 309 arestas finais em uma análise comparativa com fins demonstrativos, cujo objetivo é delimitar o patamar máximo de compressão quando a restrição $E' \subseteq E$ é relaxada. Apesar da compressão superior, a estratégia de anéis pode introduzir arestas sintéticas em outros grafos e, portanto, não garante $E' \subseteq E$ em todos os casos. Por isso, ela não deve ser interpretada como redução transitiva estrita, e seus resultados não são comparáveis diretamente às variantes válidas. A variante --reduce-dag, por sua vez, chegou a 299 arestas finais, porém sobre uma instância acíclica distinta; sua inclusão no gráfico serve apenas para referência de escopo, não para comparação direta com os métodos aplicados ao grafo cíclico.

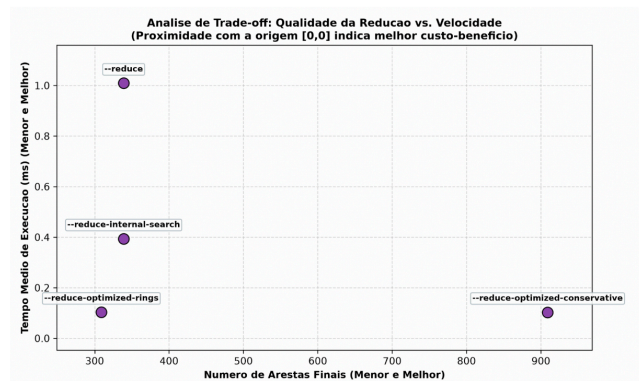


Figure 3. Análise de *trade-off*: qualidade da redução versus velocidade.

O Gráfico 3 sintetiza o compromisso entre velocidade e qualidade da redução. O --reduce funciona como referência de correteude, mas paga o maior custo computacional. A abordagem conservadora é rápida, porém pouco agressiva na remoção de arestas. A variante --reduce-internal-search apresenta o resultado mais

competitivo entre as soluções válidas para grafos gerais: preserva a atingibilidade, mantém $E' \subseteq E$ e reproduz a mesma quantidade de arestas finais do baseline com tempo significativamente menor.

Em todas as variantes válidas avaliadas, a comparação de atingibilidade entre o grafo original e o grafo reduzido indicou 0 divergências. Portanto, nas instâncias testadas, as remoções não alteraram os conjuntos de vértices alcançáveis a partir de cada origem. Esse resultado confirma que a análise não se limita à quantidade de arestas removidas: a redução só é considerada satisfatória quando combina compressão, desempenho e preservação integral da alcançabilidade.

6 Conclusão

Neste trabalho, apresentamos um estudo estrutural e um conjunto de abordagens algorítmicas para o problema da redução transitiva em grafos direcionados, combinando condensação de componentes cíclicos e heurísticas locais. Os resultados experimentais demonstram que a redução baseada em restrição de escopo por componente e a Heurística de Busca Interna, relacionada ao problema MEG (*Minimum Equivalent Graph*), superam substancialmente os métodos tradicionais de varredura, como a força bruta, em eficiência computacional, alcançando elevada compressão sem comprometer a atingibilidade sistêmica. Em particular, a variante `--reduce-internal-search` obteve a mesma quantidade de arestas finais do *baseline* na instância com SCCs, mas com menor tempo médio de execução, preservando a condição $E' \subseteq E$.

Ademais, o método proposto provê uma redução significativa na dimensionalidade das arestas, tornando o processamento de redes complexas mais gerenciável. A validação empírica indicou 0 divergências de atingibilidade nas variantes válidas avaliadas, reforçando que a remoção de arestas não alterou os conjuntos de vértices alcançáveis a partir de cada origem. Ainda assim, é importante destacar que, em grafos direcionados gerais com ciclos, a busca interna deve ser interpretada como uma heurística prática de redução, e não como uma garantia de minimização global para todos os casos.

No entanto, uma limitação teórica deste estudo reside no *trade-off* exigido para manter a otimização em tempo polinomial, limitando o escopo principal a estruturas direcionadas. A abordagem de anéis, embora apresente maior compressão nos experimentos demonstrativos, deve ser entendida apenas como contraponto experimental, pois pode introduzir arestas sintéticas e, nesse caso, deixa de satisfazer a premissa de subgrafo estrito, na qual E' deve ser subconjunto de E .

Conforme discutido, a transposição desses métodos para grafos não direcionados exige uma adaptação conceitual. Nesse cenário, preservar atingibilidade equivale a preservar conectividade entre vértices de uma mesma componente conexa, de modo que o problema se aproxima da construção de uma floresta geradora, ou de uma árvore geradora mínima quando houver pesos associados. Assim, o caso não direcionado não corresponde diretamente à mesma semântica da redução transitiva em grafos direcionados.

Finalmente, este trabalho contribui para o desenvolvi-

mento de rotinas de simplificação estrutural em Ciência da Computação, podendo ser aplicado a domínios práticos como otimização de rotas e limpeza de topologias de dependência de software. Oportunidades para pesquisas futuras incluem a implementação prática dessas heurísticas em grafos temporais, isto é, dinâmicos, e a investigação de sua eficácia em bancos de dados de larga escala. Em conclusão, a abordagem de condensação associada a restrições de escopo por componente consolida-se como um caminho promissor para pesquisas futuras em otimização de redes, possuindo potencial para ser estendida e aprimorada em estudos subsequentes.

References

- Aho, A. V., Garey, M. R., and Ullman, J. D. (1972). The transitive reduction of a directed graph. *SIAM Journal on Computing*, 1(2):131–137. DOI: 10.1137/0201008.
- Khuller, S., Raghavachari, B., and Young, N. (1995). Approximating the minimum equivalent digraph. *SIAM Journal on Computing*, 24(4):859–872. DOI: 10.1137/S0097539793256685.
- Tarjan, R. E. (1972). Depth-first search and linear graph algorithms. *SIAM Journal on Computing*, 1(2):146–160. DOI: 10.1137/0201010.